ECE420 Final Project Report
Foreground and Background Segmentation for Object Detection

Samina Bharmal, Himani Sharma
{saminab2, himanis2} @ illinois.edu

I. Introduction

This project aims to use adaptive foreground and background segmentation to separate moving objects from stationary objects. The concept of background segmentation has a variety of applications in image and video processing. We want to implement this algorithm in our project because it provides an efficient way to classify objects as background or foreground based on their pattern of motion. The fact that this method uses clusters to represent pixels instead of an average value helps it stand out from other backgrounds, because it can be used on a pseudostationary background unlike other algorithms that only target stationary background.

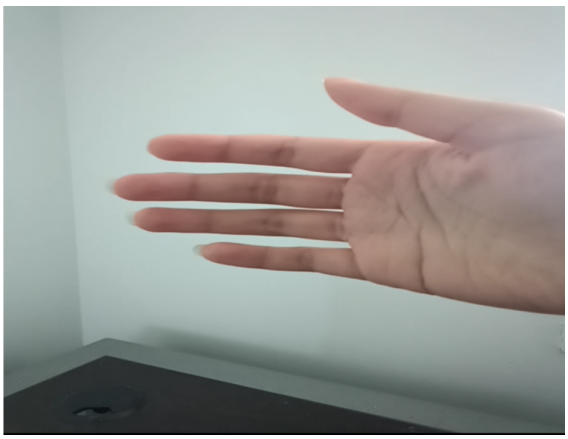

Fig. 1. Original Image.

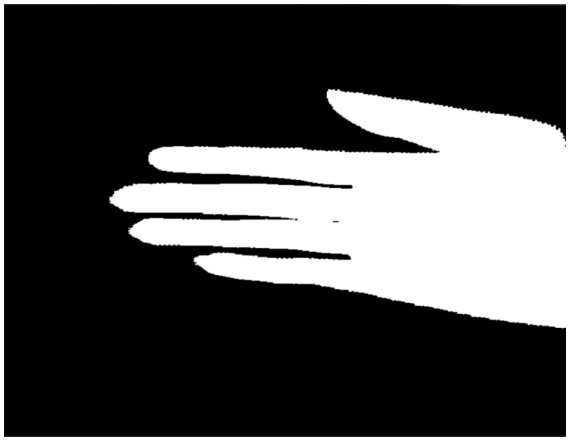

Fig. 2. After Segmentation and Pre/Post Processing.

Our final project will implement this algorithm to create a background swapping application. The user will be able to access features such as background blurring, changing the background color to a selected value, and changing the background to a separate image. The issue we are targeting with this application is background swapping when the background is not completely stationary, and may have some lighting changes. Our app will be able to handle these challenges, while working in real time to provide video output.

II. Literature Review

Foreground and background segmentation is a fundamental computer vision task that separates moving foreground objects from a pseudo stationary background. The naive approach is to use background subtraction, where the first frame is taken as the background, and any subsequent frames are subtracted from it before being thresholded for classification purposes. However, this method does not account for lighting changes, shadows, and noise in the background. Current approaches include other models like the Gaussian Mixture Model(GMM). These models represent each pixel using multiple distributions instead of an average value, which accounts for the small repetitive changes in the background. Another such technique is K means clustering, which uses centroids and weights that update every frame to model each pixel. Additionally, more advanced foreground segmentation algorithms include sample based approaches such as ViBe, and texture based methods such as SuBSENSE. These approaches generally incorporate spatial and temporal consistency to reduce flickering and noisy segmentation outputs. Other optimizations include post processing filters such as Gaussian smoothing, morphological opening and closing, majority filtering, and connected component analysis to remove isolated noisy pixels and smooth the boundary edges. Recent developments in this area include deep learning approaches that rely on convolutional neural networks. These methods can reach very high accuracies, however, they require greater computational resources and large amounts of training data. Therefore, adaptive clustering and statistical background modeling methods are used in many real time and embedded systems due to their balance between computational efficiency and segmentation performance.

III. Technical Description

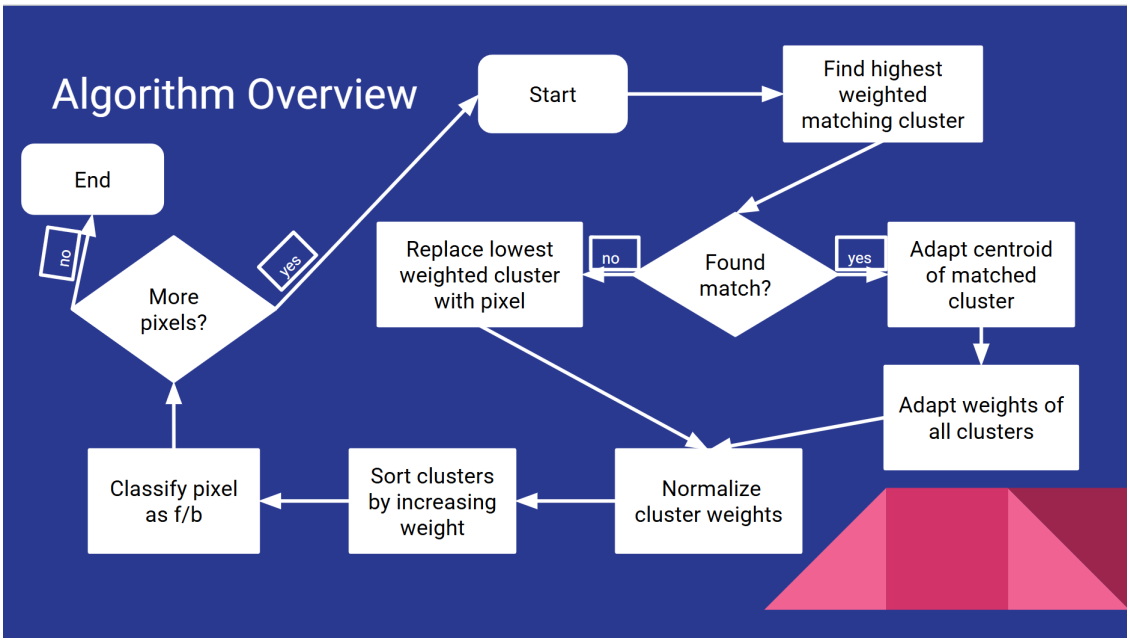

Fig. 3. Flowchart with Algorithm Overview.

In order to observe the algorithm in action, the following matrix will be used to represent the result after each step.

$$\begin{bmatrix} 100 & 200 \\ 130 & 150 \end{bmatrix}$$

In this particular example, there are three clusters where each has a weight w and centroid c.

$$C_1 = (w = 0.5,\ c = 100)$$

$$C_2 = (w = 0.3,\ c = 150)$$

$$C_3 = (w = 0.2,\ c = 200)$$

The parameters used are below.

$$\text{threshold} = 25, \text{L} = 30, \text{classification threshold} = 0.5$$

- Step 1: Cluster Matching

  This is done by searching the cluster weights in order of descending weight and comparing the manhattan distance for each one. If the manhattan distance is below the threshold value, then it is considered a match.

  For the example matrix:

$$|130 - 150| = 20 < 25$$

  So pixel 130 matches cluster C2.

  If no cluster is found, then the cluster with the minimum weight is replaced by a new cluster with the current pixel value as the centroid and a low initial weight.

$$\begin{bmatrix} 100 \to C_1 & 200 \to C_3 \\ 130 \to C_2 & 150 \to C_2 \end{bmatrix}$$

- Step 2: Adaptation

  The weights of all clusters according to this equation, which increases the weight of the matched cluster:

$$w_k^* = \begin{cases} w_k + \frac{1}{L}(1 - w_k), & k = M_k, \\ w_k + \frac{1}{L}(0 - w_k), & k \neq M_k, \end{cases} \tag{1}$$

  The centroid of the matching cluster is then updated to move closer to the pixel value that it was matched to:

$$c_k^* = c_k + \frac{1}{L}(x_t - c_k) \tag{2}$$

  For the example when cluster C2 is matched to pixel 130:

$$c_2^* = 150 + \frac{1}{30}(130 - 150)$$

$$c_2^* = 150 - \frac{20}{30}$$

$$c_2^* \approx 149.33$$

After applying these equations on the example matrix the result is:

$$C_1 = (0.4833,\ 100)$$

$$C_2 = (0.3233,\ 149.33)$$

$$C_3 = (0.1933,\ 200)$$

- Step 3: Normalization

  Since the weight of a cluster increases as a pixel is matched to it, the weights must be normalized to represent the probability that a certain cluster matches the background

  The following equation is used:

$$\tilde{w}_k = \frac{w_k}{S}, \quad \forall k, \text{ where } S = \sum_k w_k \tag{3}$$

  After applying this step on the example matrix, all of the clusters weights add up to 1.

$$C_1 = 0.483$$

$$C_2 = 0.323$$

$$C_3 = 0.193$$

  - Step 4: Classification

    The clusters weights higher than the matched cluster weight are added up according to this equation:

$$P = \sum_{k>M_k}^{K-1} w_k \tag{4}$$

    Larger values of P mean the pixel is part of the foreground and smaller values of P indicate that the pixel lies in the background.

    For the example matrix, the final result is:

$$\begin{bmatrix} 0 & 255 \\ 0 & 0 \end{bmatrix}$$

    The background pixels are represented by 0 and the foreground pixels are represented by 255. Only pixel 200 is classified as foreground, while the rest of the pixels are labeled as background.

IV. Pre/Post Processing Optimizations

In order to reduce noise and improve output quality, a pre processing algorithm and two post processing algorithms were implemented. Additionally, we added an optimization by adding two learning rates to separate the speed at which the centroid color updates from the cluster weight updates.

Segmentation Without Pre Processing

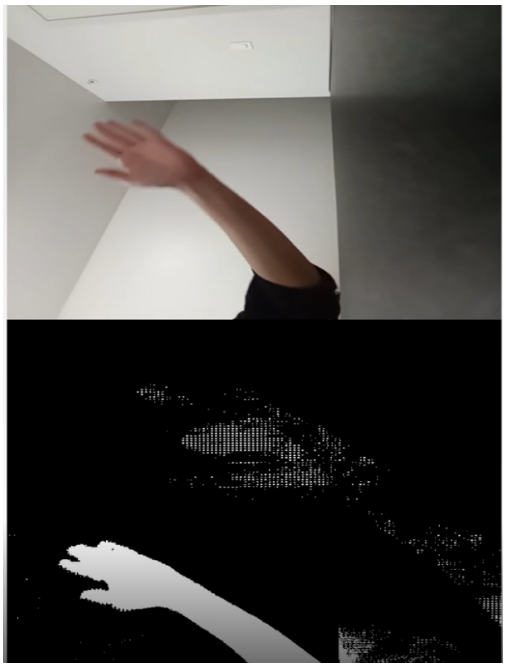

Fig. 4. Background Segmentation Without Pre Processing.

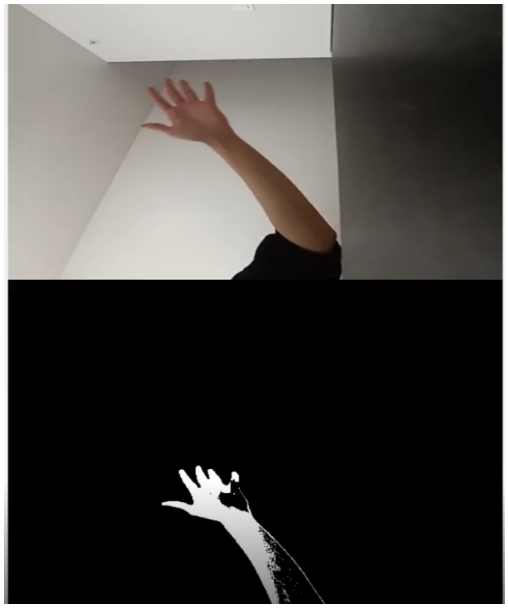

Fig. 5. Background Segmentation with Pre Processing Gaussian Filter.

– Gaussian Blur Pre Processing

Segmentation Pre Processing Gaussian

The goal of this filter is to reduce sensor noise and pixel fluctuations before segmentation. We applied a 3×3 Gaussian filter on the luminance (Y) channel:

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

This reduced false positives, improved segmentation stability, and smoothed noisy pixel transitions.

– Connected Components Analysis Post Processing

Segmentation Post Processing CCA

In order to remove small noisy foreground blobs, we applied a BFS-based 8-connected component search and removed regions smaller than the threshold size. This reduced false positives, improved segmentation stability, and smoothed noisy pixel transitions. The impact was that the foreground masks became cleaner, major objects were preserved, and isolated detections were eliminated.

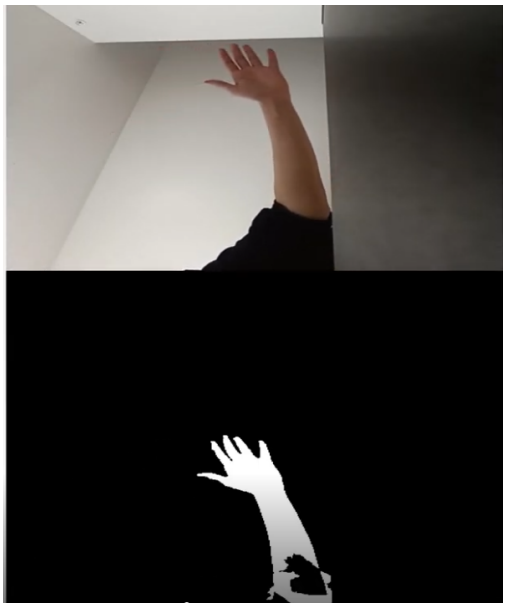

Fig. 6. Background Segmentation with Post Processing CCA Filter.

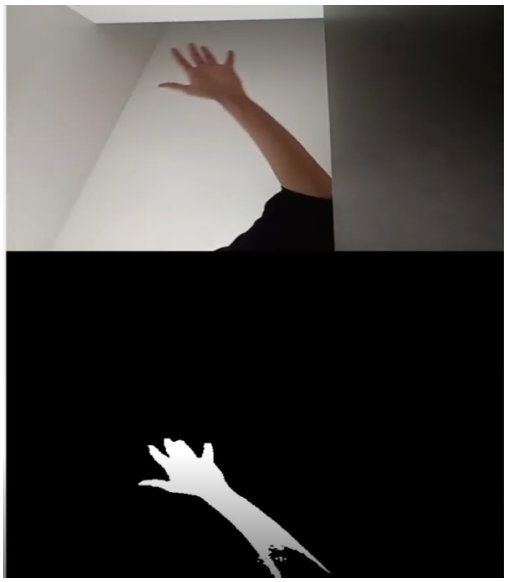

Fig. 7. Background Segmentation with Post Processing Majority Filter.

– Majority Filter Post Processing

Segmentation Post Processing Majority

To improve local consistency of segmented regions we used 3×3 neighborhood voting. If greater than 5 white neighbors were counted the pixel was set to the foreground, and otherwise it was set to the background. This filter smoothed edges, filled small holes, and reduced isolated noise pixels.

– FPS Comparison of Filters

The gaussian and majority filters have similar computation times. However, the CCA algorithm has the lowest FPS on average since it implements BFS search algorithms and utilizes stacks and heaps. This increases computation time significantly.

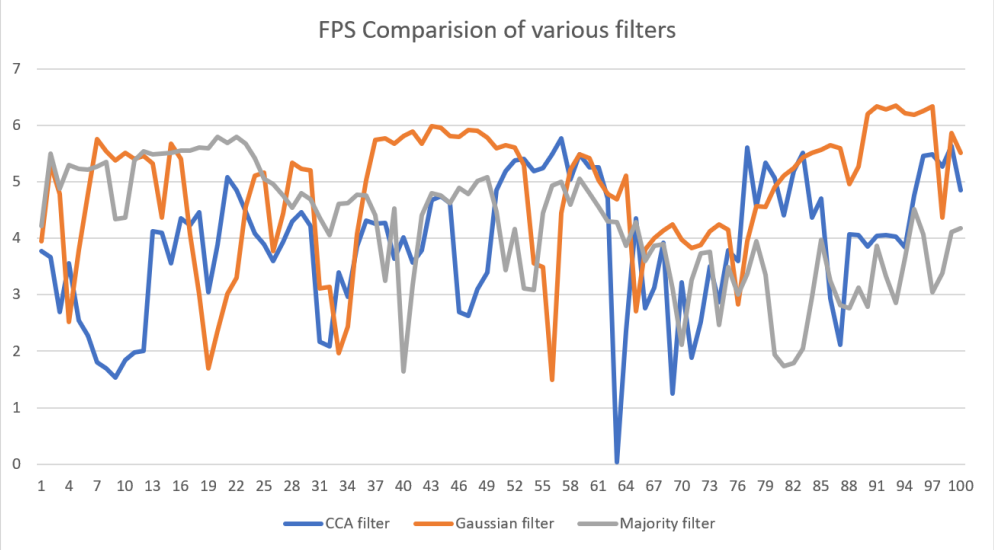

Fig. 8. FPS Comparison of Each Pre/Post Processing Filter

– Adaptive Learning Rate Optimization

In order to reduce noise without absorbing the foreground into the background, we implemented 2 different learning rates. L controls how quickly the model updates background probabilities when it is higher, the background adapts to scene changes more slowly. Lc controls how quickly color centroids update over time and when it is higher, there are smoother color transitions with moving objects. This allowed for a better handling of dynamic backgrounds because stability improved and false positives decreased (results are shown in Section V).

V. Software Documentation

- **activity_main.xml** : XML layout file containing the start page user interface for the application.
- **activity_camera.xml** : XML layout file contains the main application interface for real time foreground and background segmentation.
- **MainActivity.java** : Contains the main activity of the Android application. This file initializes the user interface by requesting camera permission and then locks the application orientation to portrait mode. Finally it calls the CameraActivity.java file and implements its functions.
- **CameraActivity.java** : Contains the main algorithm for segmentation. This file initializes the camera preview, processes incoming frames in real time and applies the segmentation algorithm. The file also contains preprocessing and postprocessing methods for noise reduction and segmentation stabilization. The function descriptions are given below:
  - `onCameraFrame()`: Processes each incoming camera frame, applies foreground background segmentation and renders the processed output onto the display canvas.
  - `yuv2rgb()`: Converts YUV image data into RGB format for visualization and applies the selected background replacement color.
  - `gaussianBlurY()`: Function that implements Gaussian blur preprocessing to the luminance channel
  - `backSeg()`: Implements the adaptive foreground and background segmentation algorithm
  - `majorityFilter()`: Implements majority filter as the postprocessing filter
  - `postProcessing()`: Performs the connected component analysis.

VI. App Layout

The layout includes several colors on the bottom to replace the background color. The top video screen contains the original video and the bottom video screen has the segmented background output. The bottom also contains a camera switch button to toggle between foreground and background.

Fig. 9. Default App Layout.

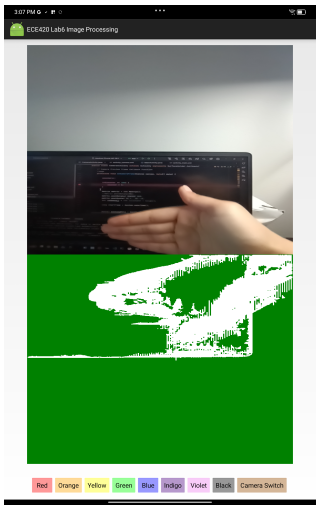

Fig. 10. Green Background Replacement.

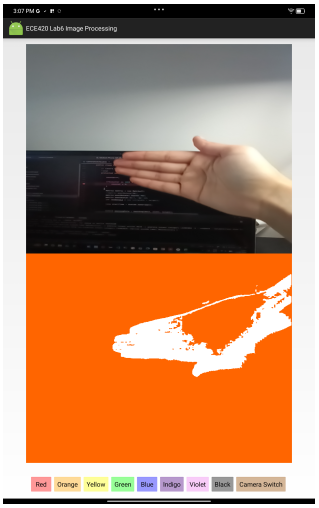

Fig. 11. Orange Background Replacement.

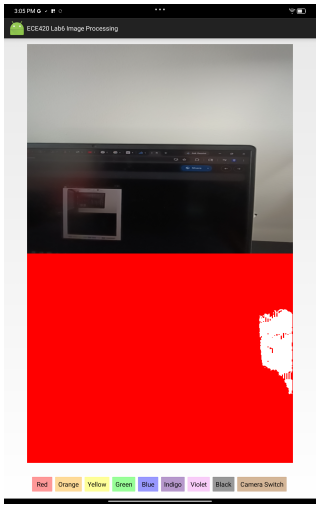

Fig. 12. Red Background Replacement.

VII. Final Results and Testing

The algorithm was tested under different lighting conditions and the parameters were tuned to eliminate background noise, completely fill in foreground silhouette, and avoid absorbing the foreground into the background.

The main challenge with this implementation was removing all white noise in the background, while not absorbing the foreground into the background too quickly. The optimization with the 2 L parameters made this tradeoff between aggressive and light background segmentation easier because it allowed background weights and centroid colors to be updated at different rates.

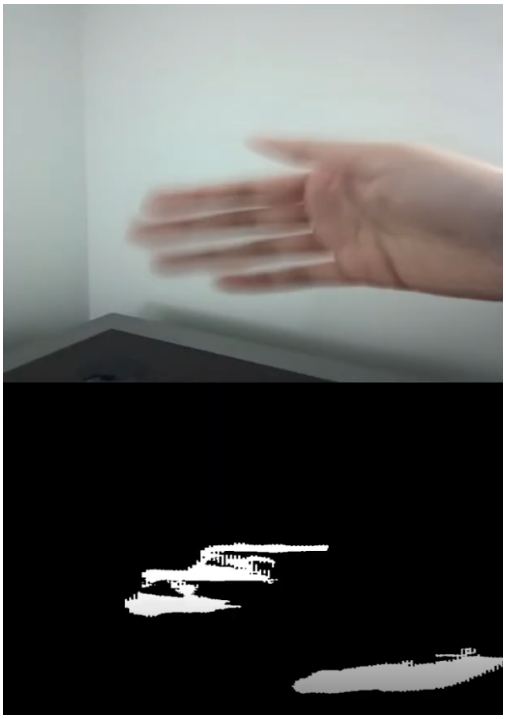

Fig. 13. Aggressive Background Segmentation.

Fig. 14. Light Background Segmentation.

Aggressive Background Segmentation

Light Background Segmentation

After testing in dark and bright rooms, the best balance between values was found with the following parameters:

$$\mathrm{K} = 3, \mathrm{L} = 30, \mathrm{Lc} = 90, \text{threshold} = 25, \text{classification threshold} = 0.5, \text{min weight} = 0.05,$$

In order to quantify the accuracy and speed of our implementation, we compared our results to the OpenCV Background Subtractor MOG2 algorithm. In the following video our algorithm is on the bottom video screen, and the OpenCV algorithm is on the top screen.
OpenCV MOG2 Comparison Video
Both algorithms were tested concurrently with the same camera input in order to measure the false positives and false negatives in relation to OpenCV MOG2, as well as the speed of each algorithm.

Fig. 15. Testing with OpenCV MOG2 as Baseline Comparison.

The resulting false positive rate for our algorithm compared to OpenCV was 13-15%. However, this can be attributed to the fact that our algorithm aims to completely fill in foreground silhouette with white, whereas the OpenCV MOG2 algorithm has many holes in the foreground.

The false negative rate was 4-6% meaning that our algorithm only had a small amount of background noise compared to OpenCV MOG2. By comparing these metrics, we are able to confirm that our algorithm is functional and accurate.

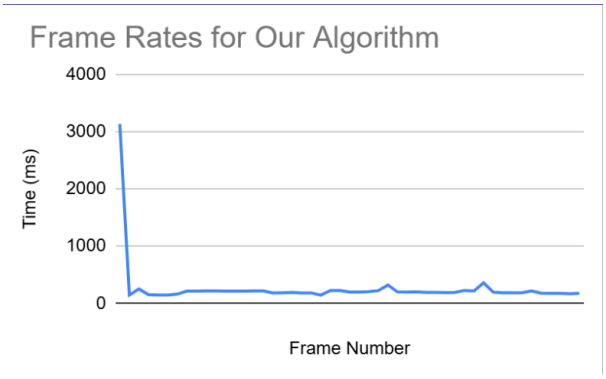

Fig. 16. Graph of FPS for Our Algorithm.

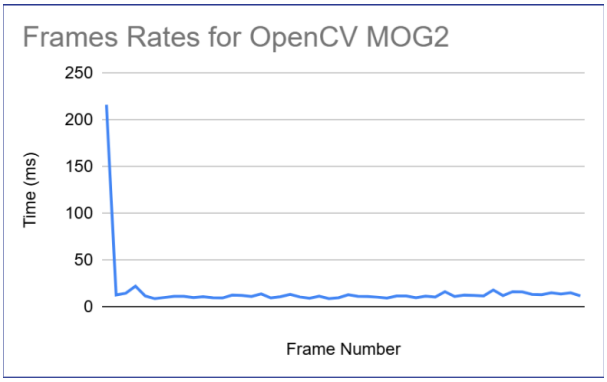

Fig. 17. Graph of FPS for OpenCV MOG2.

While the Android version of the code is 20 times faster than the python version, our algorithm is slower than OpenCV MOG2 due to the added complexity of the filters. Our algorithm has an FPS of 5, and the OpenCV MOG2 algorithm has an FPS of 80. However, our algorithm operates in real time and is responsive with a rapidly changing foreground.

VIII. Suggestions for Extensions and/or Modifications

Further extensions and modifications can be implemented to improve the quality of the foreground and background segmentation algorithm.

One such extension could be a cursor tracking application. The video below demonstrates the foreground and background segmentation algorithm operating in a real-time cursor tracking application, where the detected foreground object is tracked and used to control cursor movement on the screen. Applications of this technology include gesture based control systems, gaming, human-computer interaction, accessibility tools, virtual reality interfaces, and touchless user interaction systems.

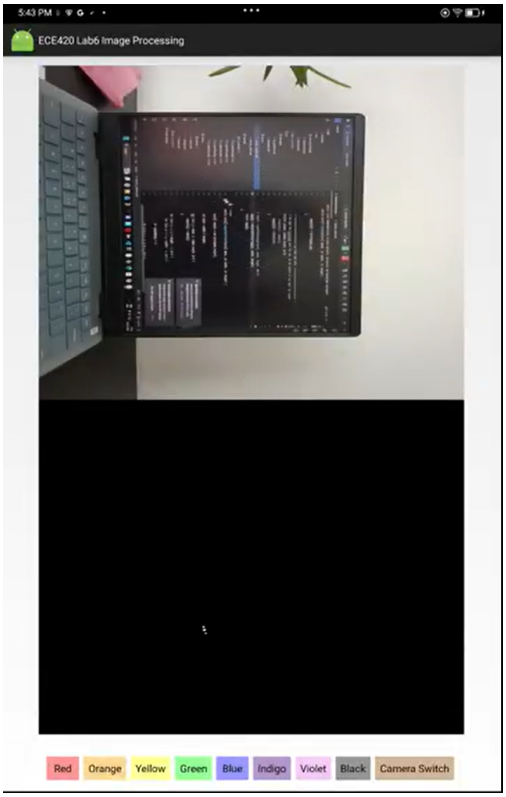

Fig. 18. Cursor Tracking

Cursor Tracking Video

Another extension can be to replace the hard threshold-based cluster matching with a probabilistic or soft-assignment approach. In the current implementation, a pixel either matches a cluster or does not match it based on a fixed distance threshold which leads to

rapid cluster switching when pixel values fluctuate even slightly. A soft-assignment method could then be used to provide smoother transitions and reduce segmentation instability. These would be based on weighted probabilities or Gaussian likelihoods.

Spatial consistency can also be improved by including neighborhood information into the classification process. Right now, each pixel is processed independently, making the algorithm sensitive to isolated noise. Markov Random Field (MRF) or Conditional Random Field (CRF) based spatial regularization techniques would make the pixels more consistent with the neighboring pixels.

Further improvements may include adaptive learning rates and dynamic thresholds. In the current implementation, the learning parameter and classification thresholds remain fixed throughout execution. However, different regions of the image require different learning rate adaptation speeds depending on motion activity or lighting variation. Adaptive parameter selection would tune the learning rates automatically based on the changing lighting conditions. Finally, deep learning based segmentation approaches may be explored for future work.

REFERENCES

[1] D. E. Butler, V. M. Bove, and S. Sridharan, "Real-time adaptive foreground/background segmentation," *EURASIP Journal on Advances in Signal Processing*, vol. 2005, no. 13, pp. 2099–2104, 2005.

[2] Wikipedia Contributors, "Markov random field," *Wikipedia, The Free Encyclopedia*. [Online]. Available: https://en.wikipedia.org/wiki/Markov_random_field [Accessed: 13-May-2026].